Secure Exit

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

SANGEPU VENKATA PRAVEEN
192373013RD

CO3-AT3-Assignment

← Back

QUESTIONS

Q1 ✓
Banking Transaction System – Exception Handling
10 marks

Q2 ✓
Hospital Patient Registration – User-Defined Exception
10 marks

Q3 ✓
E-Commerce Order Processing – Built-in Exceptions
10 marks

Q4 ✓
Hospital Emergency System – Thread Priority
10 marks

Q5 ✓
Online Food Delivery – Multiple Threads and Execution Order
10 marks

5/5 answered

Q1 Banking Transaction System – Exception Handling — 10 marks

Develop a Java program for an ATM withdrawal system. The program accepts the account balance and withdrawal amount.

The program must:

• Accept the current account balance and withdrawal amount.

• Throw a user-defined InsufficientBalanceException when the withdrawal amount exceeds the balance.

• Handle withdrawal amounts that are zero or negative.

• Handle invalid numeric input using a suitable built-in exception.

• Display a meaningful transaction result.

• Use try-catch-finally so that the transaction completion message is always displayed.

Sample Test Cases:

Input / Scenario Expected Result Purpose

10000 3000 Normal case

10000 12000 Boundary / exception case

10000 -500 Invalid input

abc 3000 Invalid input

Your Code

```
1 class Main{
```

```
    public static void main(String[] args)
        double balance = 10000,amount = 3(
        try{
            if(amount <= 0) throw new Ille
            if(amount > balance) throw new
        } catch (Exception e){
            System.out.println(e.getMessa(
        } finally {
            System.out.println("Transactio
        }
    }
}
class InsufficientBalanceException extends
    InsufficientBalanceException(String s)
        super(s);
    }
}
```

Input

```
10000
3000
```

Output

```
Transaction completed
```

Submitted

Secure Exit

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

SANGEPU VENKATA PRAVEEN
192373013RD

CO3-AT3-Assignment

← Back

QUESTIONS

Q1 ✓
Banking Transaction System – Exception Handling
10 marks

Q2 ✓
Hospital Patient Registration – User-Defined Exception
10 marks

Q3 ✓
E-Commerce Order Processing – Built-in Exceptions
10 marks

Q4 ✓
Hospital Emergency System – Thread Priority
10 marks

Q5 ✓
Online Food Delivery – Multiple Threads and Execution Order
10 marks

5/5 answered

Q2 Hospital Patient Registration – User-Defined Exception

10 marks

Develop a Java program for a hospital patient registration system. The program accepts the patient's name, age, and body temperature.
Create suitable user-defined exception classes for invalid age and invalid temperature. Validate the input before registration. Use try-catch-finally and display a registration completion message from finally.
Sample Test Cases:
Input / Scenario Expected Result Purpose
Arun, 35, 98.6 Patient registered successfully Normal case
Meena, 12, 98.4 InvalidAgeException Boundary / exception case
Ravi, 45, 105.2 InvalidTemperatureException Invalid input
Kumar, -2, 98.1 InvalidAgeException Invalid input

Your Code

```
1 class Main{
2     public static void main(String[] args)
3         int age = 35;
4         double temp = 98.6;
5         try{
6             if(age < 18) throw new Invalic
```

```
            if(temp < 95 || temp > 100) th
            System.out.println("Patient re
        } catch (Exception e){
            System.out.println(e.getMessag
        } finally {
            System.out.println("Registatio
        }

    }
}
class InvalidAgeException extends Exceptio
    InvalidAgeException(){
        super("InvalidAgeException");
    }
}
class InvalidTemperatureException extends
    InvalidTemperatureException(){
        super("InvalidTemperatureExceptio
    }
}
```

Input

```
Arun 35 98.6
```

Output

```
Patient registerd successfully
Registation completed
```

Submitted

Secure Exit

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

SANGEPU VENKATA PRAVEEN
192373013RD

CO3-AT3-Assignment

← Back

QUESTIONS

Q1 ✓
Banking Transaction System – Exception Handling
10 marks

Q2 ✓
Hospital Patient Registration – User-Defined Exception
10 marks

Q3 ✓
E-Commerce Order Processing – Built-in Exceptions
10 marks

Q4 ✓
Hospital Emergency System – Thread Priority
10 marks

Q5 ✓
Online Food Delivery – Multiple Threads and Execution Order
10 marks

5/5 answered

Q3 E-Commerce Order Processing – Built-in Exceptions — 10 marks

Develop a Java program for an online shopping application. Store product names and prices in arrays and accept a product index and quantity from the customer.

The program must:

• Calculate total cost as quantity × price.
• Handle invalid product indices.
• Handle invalid numeric input.
• Reject zero or negative quantities.
• Use appropriate built-in exception classes.
• Continue execution after handling an invalid transaction.

Sample Test Cases:

Input / Scenario Expected Result Purpose

1, 2 Valid product; total cost calculated Normal case

5, 2 ArrayIndexOutOfBoundsException handled Boundary / exception case

2, -1 Invalid quantity Invalid input

abc, 2 InputMismatchException handled Invalid input

Your Code

```
1 class Main{
2     public static void main(String[] args)
3         int index = 1,qty = 2;
4         double[] price = {100,200,300};
```

```
5           try{
6               if(qty <= 0) throw new Illega
7               System.out.println("Total cost
8           } catch (ArrayIndexOutOfBoundsExce
9               System.out.println("Invalid pr
10          } catch (IllegalArgumentException
11              System.out.println(e.getMessag
12          }
13      }
14  }
```

Input

```
1
2
100 200 300
```

Output

```
Total cost: 400.0
```

Submitted

Secure Exit

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

SANGEPU VENKATA PRAVEEN
192373013RD

CO3-AT3-Assignment

← Back

QUESTIONS

Q1 ✓
Banking Transaction System – Exception Handling
10 marks

Q2 ✓
Hospital Patient Registration – User-Defined Exception
10 marks

Q3 ✓
E-Commerce Order Processing – Built-in Exceptions
10 marks

Q4 ✓
Hospital Emergency System – Thread Priority
10 marks

Q5 ✓
Online Food Delivery – Multiple Threads and Execution Order
10 marks

5/5 answered

Q4 Hospital Emergency System – Thread Priority — 10 marks

A hospital application creates threads for Emergency Patient Processing, General Patient Processing, and Billing Processing.

Develop a Java program that assigns different priorities to these threads. Run the program several times and analyze whether the emergency thread is guaranteed to execute first.

Then modify the design if the hospital requires a strict execution sequence rather than merely expressing scheduling preference.

Your answer must contain Java code and an execution analysis based on the actual thread behavior.
Sample Test Cases:
Input / Scenario Expected Result Analysis Focus
Emergency, General, Billing All threads execute; priority does not guarantee a fixed output order
Normal execution
Emergency workload > General workload Emergency thread receives a higher priority but exact ordering is not guaranteed Concurrency / design
Strict emergency-before-billing requirement Use

explicit coordination rather than relying only on priority Exception / boundary

Your Code

```
class Main{
    static void order(){
        System.out.println("Order Processi
    }
    static void payment(){
        System.out.println("Payment Proces
    }
    static void delivery(){
        System.out.println("Delivery Assig
    }
    public static void main(String[] args)
        Thread t1 = new Thread(() -> order
        Thread t2 = new Thread(() -> payme
        Thread t3 = new Thread(() -> deliv
        t1.start();
        t1.join();
        t2.start();
        t2.join();
        t3.start();
        t3.join();

    }
}
```

Input

stdin input

Output

```
Order Processing
Payment Processing
Delivery Assignment
```

Submitted

Secure Exit

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

SANGEPU VENKATA PRAVEEN
192373013RD

CO3-AT3-Assignment

← Back

QUESTIONS

Q1 ✓
Banking Transaction System – Exception Handling
10 marks

Q2 ✓
Hospital Patient Registration – User-Defined Exception
10 marks

Q3 ✓
E-Commerce Order Processing – Built-in Exceptions
10 marks

Q4 ✓
Hospital Emergency System – Thread Priority
10 marks

Q5 ✓
Online Food Delivery – Multiple Threads and Execution Order
10 marks

5/5 answered

Q5 Online Food Delivery – Multiple Threads and Execution Order — 10 marks

An online food-delivery application uses three threads: Order Processing, Payment Processing, and Delivery Assignment.

Develop a Java program in which these activities are represented by separate threads. Analyze the problem if all three threads are started independently.

Modify the program so that:
• Order processing completes before payment processing.
• Payment processing completes before delivery assignment.
• The program uses appropriate Java thread-management techniques.
• The output clearly shows the required execution order.

Explain the difference between simply starting all threads and explicitly controlling their execution order.
Sample Test Cases:
Input / Scenario Expected Result Analysis Focus
Order=Pizza; Payment=UPI; Delivery=Agent A Order → Payment → Delivery Normal execution
Order=Burger; Payment=Card; Delivery=Agent B

Order → Payment → Delivery Concurrency / design Payment=Failed Delivery assignment should not proceed Exception / boundary

Your Code

```
class Main{
    static String b;
    static synchronized void produce(Strin
        while (b!=null){
            Main.class.wait();
        }
        b = s;
        System.out.println("Produced " + s
        Main.class.notifyAll();
    }
    static synchronized void consume() thr
        while (b == null){
            Main.class.wait();
        }
        System.out.println("Consumed " + b
        b = null;
        Main.class.notify();
    }
    public static void main(String[] args)
        Thread t1 = new Thread(() -> {
            try {
                produce("P1");
                produce("P2");
                produce("P3");
            } catch (Exception e) {
                System.out.println(e.getMe
            }
        });
        Thread t2 = new Thread(() -> {
            try {
                consume();
                consume();
                consume();
            } catch (Exception e){
                System.out.println(e.getMe
            }
        });
        t1.start();
        t2.start();
        t1.join();
        t2.join();
```

```
42      }
43  }
```

Input

```
stdin input
```

Output

```
Produced P1
Consumed P1
Produced P2
Consumed P2
Produced P3
Consumed P3
```

Submitted